EDB Creator API

This document describes the API to create and fill an EDB (Electric Wiring Database) and how to save and restore it to/from a file. The API is available for the C , Tcl , Java , Python , and .NET C# languages (see below for the C API, Tcl API, Java API, Python API, and .NET C# API). For an overview and better understanding, please also check the database object model at the beginning.

The EDB Object Model

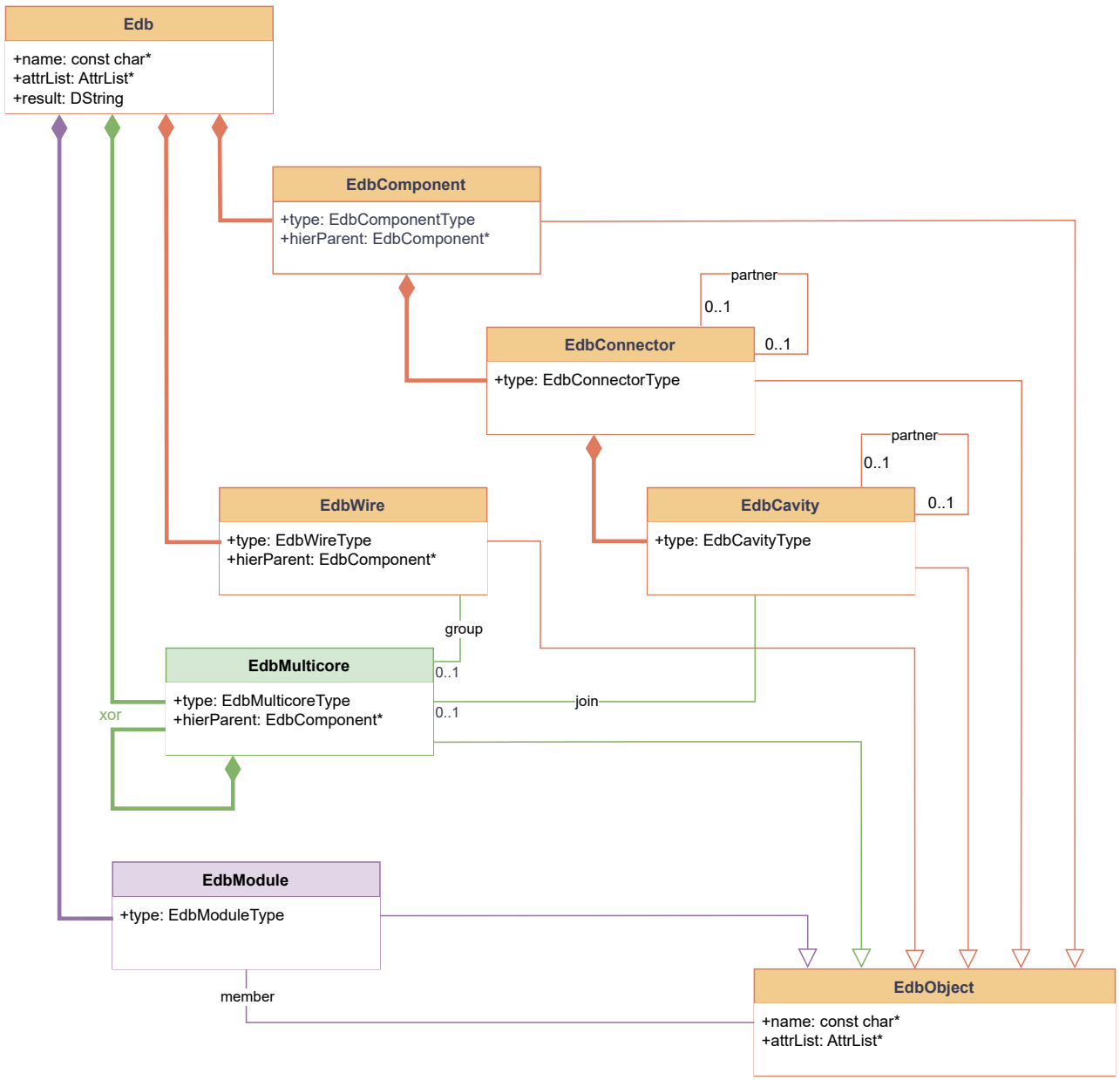


Figure 1: *The EDB object model*

The `Edb` is the database root. It manages all EDB objects and their relations. We divide the relations into "aggregation" – forming the **containment tree** – and "association" – storing the **connectivity** and other information.

The **Containment Tree:**

The database root **contains** components, wires, and multicores. Each `EdbComponent` object contains connectors, each `EdbConnector` object contains cavities, and each `EdbMulticore` object *may* contain nested multicores.

The parent relation of the EDB objects is as follows:

Object	Object's parent
`EdbComponent`	(*database root*)
`EdbConnector`	`EdbComponent`
`EdbCavity`	`EdbConnector`
`EdbWire`	(*database root*)
`EdbMulticore`	either (*database root*) or another `EdbMulticore`
`EdbModule`	(*database root*)

The Multicore's parent reference to the *database root* is technically modeled as NULL pointer.

The **Connectivity:**

The electrical **connectivity** is defined by a *n*-to-*m* association between cavities and wires. A (physical) wire usually connects to two cavities. However, in order to model logical connectivity, wires can connect to an arbitrary number of cavities. A cavity connects to zero, one, or multiple wires.

Two connectors of the same Inliner component can be set into relation (*partner*) representing the internal electrical connectivity between the left and right connector of an Inliner. Similarly, two cavities of two related connectors can also be related representing a direct electrical connection.

These relations are modeled with optional 1-to-1 associations between two connectors or two cavities, respectively, and are only supported for Inliner components.

The **Grouping of Wires:**

The wire **grouping** is defined by a 1-to-*n* association between multicores and wires. The multicore tree is part of the Containment. However, the “leaves” of the multicore tree are just references to Wires (so the Wires themselves are not part of the Multicore-Containment tree, all Wires are aggregated in the database root.

The **Modules:**

The Modules are optional. Each Module defines a (normally small) sub-set of the database by maintaining a list of references to objects of any type. Circular references between modules (e.g., when module 1 contains module 2 and vice-versa) are not allowed.

The C Language API

The EDB Creator API consists of a set of creator functions, which are defined in the C header file edbcreate.h (use `#include "edbcreate.h"` to include the header file into your C application) and the save/restore functions are defined in the C header files edbsave.h and edbrestore.h, respectively. These functions are described in the following sections:

- Building the Containment Tree Structure
- Setting StrictMode Flags
- Adding Connectivity
- Grouping Wires to a Multicore
- Details about the Inliner Component
- Defining Hierarchy
- Defining Modules
- Save to a file
- Restore from a file
- An Example Program

Object Identification

The EDB objects' internal `struct` definitions are hidden from the user. That means, only the creator functions are available and necessary to fill the database. EDB objects are identified with C-pointers to anonymous `structs`, which may be `NULL` to indicate “no object” or an error, depending on the function. The anonymous EDB `structs` are defined as follows:

```
typedef struct Edb             Edb;
typedef struct EdbComponent    EdbComponent;
typedef struct EdbConnector    EdbConnector;
```

```
typedef struct EdbCavity       EdbCavity;
typedef struct EdbWire         EdbWire;
typedef struct EdbMulticore    EdbMulticore;
typedef struct EdbModule       EdbModule;
```

The creator functions' return value is either a pointer to an EDB `struct` or a Boolean value. In case of an error, the functions return `NULL` or `false`, respectively, and leave an error message in the EDB, which can be retrieved by calling `EdbLastError()`. The Boolean type is defined as:

```
enum EdbBool {
     EdbFalse = 0,
     EdbTrue  = 1
};
```

There is an optional second approach for object identification, that is persistent between save and restore, as described below at Object ID by Number.

Building the Containment Tree Structure

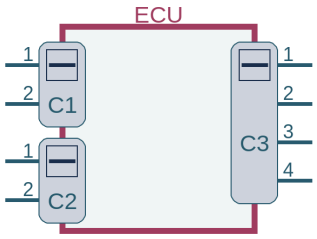
```
Edb*  EdbNew();
void  EdbDelete(Edb*);
```

The `EdbNew` function creates an empty database and returns a pointer, which identifies the database and must be passed as the first argument to all other API functions. Manipulating multiple databases simultaneously is supported.

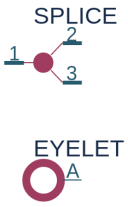
The `EdbDelete` function first deletes the database's contents and then deletes the database root, which invalidates the Edb pointer.

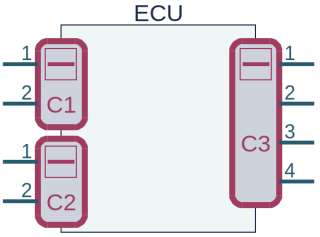
```
EdbComponent* EdbNewComponent(Edb*,                const char*, EdbComponentType);
EdbConnector* EdbNewConnector(Edb*, EdbComponent*, const char*, EdbConnectorType);
EdbCavity*    EdbNewCavityEx( Edb*, EdbConnector*, const char*, EdbCavityType);
```

The `EdbNewComponent` function creates a new Component object and adds it to the database root. The second argument specifies the component's name and may be `NULL` if there is no name to be displayed. The EdbComponentType specifies the component's type: ECU, INLINER, SPLICE, etc. A **component** includes an arbitrary number of connectors and is displayed as a rectangular box in the Nlview schematic. The INLINER component needs some extra information and is displayed in a slightly different way, as described below.

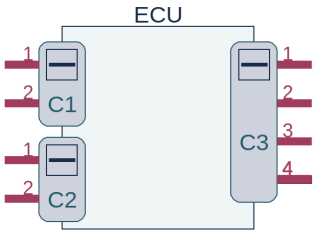


The `EdbNewConnector` function creates a new connector object and adds it to the specified component given as second argument. The connector's name is provided as third argument, and may be `NULL` if there is no name to be displayed. For an ECU or INLINER, a **Connector** models a group of connections like an electrical jack/plug respective a connector with multiple different electrical signals. In the Nlview schematic, it will be displayed as a part of a component. For an INLINER, the EdbConnectorType specifies the connector's side, that can be “UNDEF” or “ANTI” (see below). For a SPLICE or EYELET, a **Connector** models a group of electrically short-cut cavities. Eyelets and splices are expected to have exactly one connector (although it is not forbidden to have multiple connectors). They will be displayed in the Nlview schematic as dedicated (typically small) symbols.





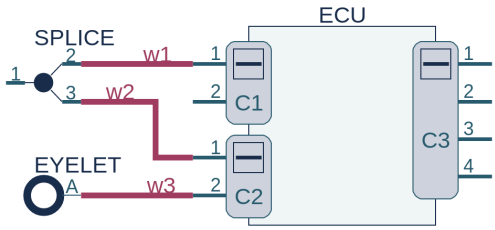
The `EdbNewCavityEx` function creates a new Cavity object and adds it to the specified Connector (given as second argument). The cavity's name is provided as third argument, and may be `NULL` if there is no name to be displayed. A **Cavity** models a Connector's connection point. Each Cavity can connect to zero, one, or multiple Wires. In the Nlview schematic, a Cavity will be displayed as a Connector pin. The EdbCavityType specifies the Cavity's appearance and behavior: UNDEF (normal), HALFDOT, SPLICED.




```
EdbWire*      EdbNewWireEx(   Edb*,                const char*, EdbWireType);
EdbMulticore* EdbNewMulticore(Edb*, EdbMulticore*, const char*, EdbMulticoreType);
```

The `EdbNewWireEx` function creates a new Wire object, which represents a 2 terminal connector (or *n* terminal connection) defining the electrical connectivity. The wire's name is provided as second argument, and may be `NULL` if there is no name to be displayed. In the Nlview schematic, wires will be displayed as polylines.

Wires of type `EdbWireTARC` play a special role. They are used to represent electrical connections between cavities of the same component, e.g., between the two cavities of a fuse. Albeit not visible in EEvision, they influence the behavior of the extractor algorithms. Note that arc wires at not possible at splices and eyelets. All cavities that are connected with an arc wire must belong to the same component. Arc wires cannot be part of a multicore.



The `EdbNewMulticore` function creates a new Multicore object and adds it to the database root if the second argument is `NULL`, or adds it to a specified parent Multicore, forming a tree. The name provided as third argument may be NULL. The EdbMulticoreType adds additional type information: TWISTED, SHIELDED, TWSHIELDED or UNDEF. A **Multicore** tree represents a nested combination of twisted and shielded wires. The TWSHIELDED is a short-cut for a SHIELDED that includes one TWISTED.

Attributes

Attributes usually act as a general means for carrying arbitrary information as name-value pairs. In EEvision, those general attributes have no direct influence on the generated schematic, but as an exception, there are some **reserved attributes**, which can optionally be used to control rendering details like colors, etc, if this feature is enabled with the "`EdiInit`" function (in the Edb VDI API). Those **reserved attributes** all start with a leading space character in their names. There is a separate documentation page "**EEvision Attributes**" with details on that.

```
EdbBool  EdbNewAttr(          Edb*,                const char*, const char*);
EdbBool  EdbNewAttr4Component(Edb*, EdbComponent*, const char*, const char*);
EdbBool  EdbNewAttr4Connector(Edb*, EdbConnector*, const char*, const char*);
EdbBool  EdbNewAttr4Cavity(   Edb*, EdbCavity*,    const char*, const char*);
EdbBool  EdbNewAttr4Wire(     Edb*, EdbWire*,      const char*, const char*);
EdbBool  EdbNewAttr4Multicore(Edb*, EdbMulticore*, const char*, const char*);
EdbBool  EdbNewAttr4Module(   Edb*, EdbModule*,    const char*, const char*);
```

The `EdbNewAttr4…` functions add attributes to the given object (second argument). Each attribute is a name-value pair. The name (third argument) must not be `NULL`, and for one object, the names should be unique among all attributes. The string memory required for the names and values is managed by the database and will be cleared when calling EdbDelete. Similar to object attributes, the `EdbNewAttr` function adds attributes to the database root.

```
const char* EdbCreateAttr( Edb*, EdbObject*, const char*, const char*);
const char* EdbUpdateAttr( Edb*, EdbObject*, const char*, const char*);
void        EdbUpdateName( Edb*, EdbObject*, const char*);
const char* EdbUpdateDupAttr(Edb*, EdbObject*, const char*, int, const char* []);
```

The `EdbCreateAttr…` function works identical to the EdbNewAttr4... functions above, except that the object is specified by the more general base class pointer `EdbObject*` (so the caller must cast the pointer type). If the given `EdbObject*` is NULL, then this function creates the attribute at the database root. The return pointer points may be NULL to indicate an error or points to the internally allocated attribute name.

The `EdbUpdateAttr…` function updates (changes) the value of the named attribute. If the attribute does not exist, then this function creates a new one as EdbCreateAttr does. The given attribute value (forth argument) may be NULL to mark this attribute as unset (to remove it). The attribute name (third argument) must not be NULL.

The `EdbUpdateName…` function is unrelated to the Attributes – it updates (changes) the Object name. The given value (third argument) may be NULL to unset the object's name (to remove it).

The `EdbUpdateDupAttr…` function updates (changes) all values of the named attribute, similar to the `EdbUpdateAttr()` function, but it addresses all values (of duplicate attributes). This function will delete all existing attribute values (with the given name in the third argument) and add the values given as vector (in the forth and fifth argument). But please note: the EDB provides only limited support for duplicate attributes with EdbUpdateDupAttr() and EdbSearchDupAttr(). The EdbUpdateAttr() and EdbSearchAttr() do not. They always address the first hit only.

```
EdbBool EdbValidateAttr(Edb*, const char*, const char*);
```

This function checks the given **reserved attribute**. It actually checks if the given value is supported by EEvision, and if not, returns an error. If the `EdbStrict`FValidAttr flag is set, then the EdbNewAttr functions above (also EdbCreateAttr, EdbUpdateAttr and EdbUpdateDupAttr) will implicitly check the reserved attribute values by calling `EdbValidateAttr()` before adding them.

For the "`expr`" attribute (Boolean expressions at CONFIG modules) there are two specials: (a) this function returns a list of all variables used in the expression (if there is no error), and (b) if this function is called with NULL value, then is validates all CONFIG module's Boolean expressions and returns a list of all variables used in all expression (if there is no error).

Setting StrictMode flags

```
void                Edb SetStrict Mode(Edb*, enum EdbStrictFlags);
enum EdbStrictFlags Edb GetStrict Mode(Edb*);
```

The EdbStrictFlags is an or-combination of individual EDB-global flags that can be set or cleared to control the behavioral of the EDB Creator API. The supported flags are `EdbStrictFPedanticSE`, `EdbStrictFIgnoreDupl`, `EdbStrictFRestoreAttr`, `EdbStrictFValidAttr`.

Adding Connectivity

```
EdbBool  Edb Join(Edb*, EdbCavity*, EdbWire*);
```

The `EdbJoin` function adds *n*-to-*m* associations between Wires and Cavities representing electrical connectivity. The Nlview software displays the connectivity by routing the Wires to the respective Connector pins. Re-joining the same Wire to the same Cavity is an error, unless the global flag `EdbStrictFIgnoreDupl` is set (then it is silently ignored). If the global flag `EdbStrictFPedanticSE` is set, then multi-wire connections at the same cavity at SPICE or EYELET are rejected with an error. The EdbSetStrictMode function sets or clears those global flags.

```
EdbBool  Edb Partner Connector(Edb*, EdbConnector*, EdbConnector*);
EdbBool  Edb Partner Cavity(   Edb*, EdbCavity*,    EdbCavity*);
```

The `EdbPartner` functions are supported only for Inliner components. They add an *1*-to-*1* association between two Connectors and between two Cavities, respectively. Nlview uses this information to correctly display the Inliner Connectors and Cavities.

Grouping Wires to a Multicore and connecting the Multicore shield

```
EdbBool  Edb Group Wire( Edb*, EdbWire*, EdbMulticore*);
EdbBool  Edb Shield Wire(Edb*, EdbWire*, EdbMulticore*);
```

The `EdbGroup` function adds a *1*-to-*n* association between one Multicore object and *n* Wires (each Wire can be a member of at most one Multicore). This relation does not change the containment tree, that is, the Wires' parent is always the database root. The `EdbShield` function does the same as the `EdbGroup` function, but in addition, it identifies this wire to represent the Multicore shield. Each Multicore with a connected shield (SHIELDED or TWSHIELDED) should have a Wire assigned with `Edb Shield Wire`. This shield-wire is used to connect the Multicore shield to one or multiple Cavities.

Object Names

Each object has an optional **name**, that can be specified as an argument to the functions `EdbNewComponent`, `EdbNewWireEx`, `EdbNewConnector`, `EdbNewMulticore` and `EdbNewCavityEx` (the name argument may be `NULL`). This object name will be sent to Nlview to be displayed as a name label at the object in the Schematic (however no label will be displayed, if the object **name** is set to NULL). The string memory required for the names is managed by the database and will be cleared when calling EdbDelete (there is also an "`EdbUpdateName`" function to change the object name later).

Object ID by Number

To optionally specify unique id numbers for the objects created by this Creator API, the Edb should first be informed about the id-range by calling `Edb InitMaxID` before loading. And then, for each object that should get a predefined id number, the `Edb SetNextID` should be called before creating the corresponding object, that means, directly before calling Edb**New**Component, Edb**New**Connector, Edb**New**CavityEx, Edb**New**WireEx, Edb**New**Multicore, or Edb**New**ModuleEx.

```
void Edb InitMaxID(Edb*, unsigned maxid);
void Edb SetNextID(Edb*, unsigned id);
```

This *id* is silently rejected if it is a duplicate or if it is out-of-range `[1…maxid)`. In these cases a unique *id* is automatically chosen. The "Query API" can be used to access those ID numbers and to look-up objects by those ID numbers.

Type Information

Each object has an optional sub-**type** information that depends on the object type (`Component`, `Connector`, `Multicore`, etc). That sub-**type** information is specified by the last argument of the corresponding creation function. The types are defined by the following C enumerators (see also edbtypes.h):

```
enum Edb Component Type {
     Edb Component T UNDEF,
     Edb Component T ECU,
     Edb Component T INLINER,
     Edb Component T SPLICE,
```

```
     EdbComponentTEYELET,
     EdbComponentTSVG,
     EdbComponentTHIER,
     EdbComponentTHBOX
};
enum EdbConnectorType {
     EdbConnectorTUNDEF,
     EdbConnectorTMALE,
     EdbConnectorTFEMALE,
     EdbConnectorTINVISIBLE,
     EdbConnectorTANTI   /* flag */
};
enum EdbCavityType {
     EdbCavityTUNDEF,
     EdbCavityTHALFDOT,
     EdbCavityTSPLICED,
     EdbCavityTIN,   /* flag */
     EdbCavityTOUT   /* flag */
};
enum EdbWireType {
     EdbWireTUNDEF,
     EdbWireTGROUND,
     EdbWireTPOWER,
     EdbWireTLOGICAL,
     EdbWireTBUS,
     EdbWireTHV,
     EdbWireTARC
};
enum EdbMulticoreType {
     EdbMulticoreTUNDEF,
     EdbMulticoreTTWISTED,
     EdbMulticoreTSHIELDED,
     EdbMulticoreTTWSHIELDED
};
enum EdbModuleType {
     EdbModuleTUNDEF,
     EdbModuleTCONFIG,
     EdbModuleTHARNESS,
     EdbModuleTSIGNAL,
     EdbModuleTFUNCTION,
     EdbModuleTALWAYS,
     EdbModuleTBUS
};
enum EdbModuleOption {
     EdbModuleOAUTOCOMPLETE
};

typedef enum EdbComponentType  EdbComponentType;
typedef enum EdbConnectorType  EdbConnectorType;
typedef enum EdbCavityType     EdbCavityType;
typedef enum EdbMulticoreType  EdbMulticoreType;
typedef enum EdbModuleType     EdbModuleType;
typedef enum EdbModuleOption   EdbModuleOption;
typedef enum EdbBool           EdbBool;
```

Inliner Component

Usually, Inliner are pretty simple Components with two Connectors that connect 1:1 from left to right (or vice versa). The picture “X2” on the right side displays such a simple Inliner Component.

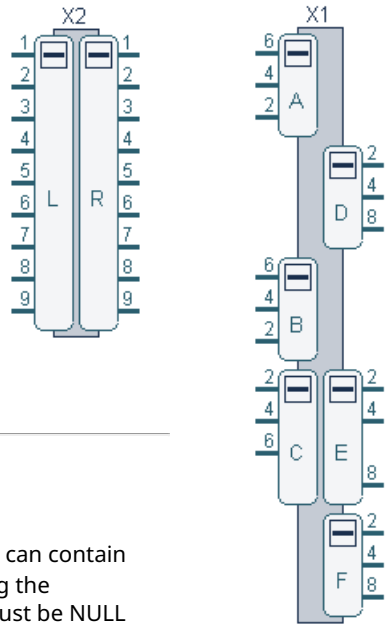
However, in certain situations, Inliner can be more complex by three reasons: (a) there may be multiple Connector pairs and (b) some of the Connectors may not be available from the current data source and (c) a Connector pair may have missing Cavities. To address (a) and (b), multiple Connector pairs and single Connectors should be grouped to appear on the left or right side. Since Nlview may flip left and right, the EDB must only specify the Connectors on one side versa those on the opposite side. The EdbConnectorType “UNDEF” or

"ANTI" should be used to define those two sides. Also the Connector pairs must be defined with the Connector Partner relation (set with EdbPartnerConnector).

To address (c) the Cavity pairs must be defined with the Cavity Partner relation (set with EdbPartnerCavity). Cavity pairs are only possible within Connector pairs and will be displayed by Nlview side by side as C2-E2 and C4-E4 in the picture "X1" on the right side. Unpaired Cavities (those without a Partner relation) will be displayed separately as C6 or E8 in the "X1".

The "X1" also displays the unpaired Connectors A, B, D and F. Also, D, E and F got the "ANTI" EdbConnectorType at creation time, so they get the "opposite side", while A, B and C get the first side.

Note that connectors at inliner components do not support the connector type "INVISIBLE".



Hierarchy

Design hierarchy is defined by hierarchy components (EdbComponentType HIER or HBOX). Both can contain other components and wires as their content. The content is added to that hierarchy after calling the **`SetHierarchy`**(*`edb, hier`*) function, where *hier* must refer to a HIER or HBOX component or must be NULL (the NULL returns back to top-level of the hierarchy).

```
void EdbSetHierarchy(Edb*, EdbComponent*);
```

The **HIER** Hierarchy:

> The HIER components behave very much like Inliner Components, defining pairs of connectors where the "ANTI" connectors are inside the hierarchy border. The hierarchy border will be displayed as a large rectangle with its contents inside the rectangle. The non-ANTI connectors are located at this border to connect wires from the outside. Hierarchy-crossing wires are not supported; that means that there usually is a wire connecting from the outside and another wire connecting from the inside. The inside wire belongs to the contents of that HIER component (and is required to be created after **`SetHierarchy`**() has been called). Nested hierarchy is supported.

The **HBOX** Hierarchy:

> The HBOX components are used as a "grouping device" and must NOT have any connectors or cavities. The hierarchy border will be displayed as a large rectangle with its contents inside the rectangle – but without any connectors at the border. Hierarchy-crossing wires are supported here; that means a wire can connect any connector outside and another connector inside the hierarchy (the inside connector belongs to a component inside the HBOX a.k.a. the contents of that HBOX).
> Nested hierarchy is supported and a wire may cross multiple HBOX borders. A hier-crossing wire must belong to the outer-most hierarchy it traverses, i.e., it can only connect to the same hierarchy or to lower hierarchy objects, provided that the lower hierarchy objects resides in a HBOX or a path of nested HBOXes.
> In other words: A hier-crossing wire must belong to the deepest HBOX (or NULL) that is a hierarchy predecessor of all components it is connected to.

A mix of HBOX and HIER hierarchy is supported, but of course, a hier-crossing wire can only cross HBOX borders.

Defining Modules

A "Module" refers to a set of Edb Objects to define a subset of the database contents. The Modules can, e.g., be used by the built-in extractor "EdbExtractByModules" or by the built-in filter "EdbFilterByModules" to display or filter the contents of a "module".

```
EdbModule* EdbNewModuleEx(Edb*, const char*, EdbModuleType);

EdbBool EdbAddObject2Module(    Edb*, EdbModule*, EdbObject*);
EdbBool EdbAddOption2Module(    Edb*, EdbModule*, EdbModuleOption);
EdbBool EdbAddObjectPair2Module(Edb*, EdbModule*, EdbObject*, EdbObject*);
EdbBool EdbAddObjectAttr2Module(Edb*, EdbModule*, EdbObject*, const char*, const char*);
```

The `NewModule` function creates a new Module and adds it to the database. The second argument specifies the module's name, that may be `NULL` if there is no name associated with the module.

The `AddObject*2Module` functions add **references** to a module. Three different reference classes are supported: (a) pointer to other Edb objects, (b) pointer-pairs, each to two other Edb objects, and (c) an object-pointer plus a name-value pair like an attribute. Here, `EdbObject*` is a pointer to a Component, Connector, Cavity, Wire, Multicore or Module. The caller must cast the pointer type. The name-value pairs, added with `AddObjectAttr2Module`, are not added to the object itself, they are only added to the **reference**.

The `AddOption2Module` function adds an **option-bit** to a module. This is usually something that changes details on the behavior of the given module. The option **AUTOCOMPLETE** means, that additional references are to be automatically added to a FUNCTION module, **after** the EDB was filtered to a 100% data model (for more accuracy). The given Edb**Module** is expected to initially include a set of Edb**Cavity** objects. The `AUTOCOMPLETE` bit will assume these Cavities as start/end points and traverse the Wires, Splices, Eyelets and Inliners to find paths between all start/end Cavities and will implicitly add those objects to the Module, too.

Saving the EDB to a file

To save an existing Edb into a file, use one of the following functions.

```
EdbBool EdbSave(Edb*, FILE* fp);
EdbBool EdbSaveFile(Edb*, const char* filename);
EdbBool EdbSaveCrypted(Edb*, const char* filename, const char* passwd, unsigned int);
```

The first function `EdbSave()` needs a FILE* to an already opened file as argument, the second `EdbSaveFile()` just the name of the file which will then be opened and closed internally. The third `EdbSaveCrypted()` additionally needs a secret password for encrypting the file contents and an "unsigned" number to identify the cipher used. Currently supported are 4 (encryption) or 5 (lz4 compression and encryption) or 7 (lz4 compression and encryption with the ChaCha20 cipher).

Restoring the EDB from a file

To restore an Edb from a file, use one of the following functions.

```
EdbBool EdbRestore(Edb*, FILE* fp);
EdbBool EdbRestoreFile(Edb*, const char* filename);
EdbBool EdbRestoreCrypted(Edb*, const char* filename, const char* passwd);
```

The first function `EdbRestore()` needs a FILE* to an already opened file as argument, the second `EdbRestoreFile()` just the name of the file which will then be opened and closed internally. The third `EdbRestoreCrypted()` additionally needs a secret password to be able read an encrypted edb file. The given database is expected to be empty. Otherwise the restored data will be added to the Edb.

Note: On Windows platforms please make sure, that any fopen() call has the "b" option set for binary open. When using the dynamic library libedbD.DLL, then you actually must use `EdbSaveFile()` and `EdbRestoreFile()` with the filename argument (because passing "`FILE*`" between DLLs is unsafe).

An Example Program

Here is an **Example Program** that creates a simple EDB and stores it to a file. That file can be restored into an EDB and displayed in Nlview. With pretty default EDI configuration settings, the Extractor “ExtractOne” should create the image in Figure 2 (or similar).

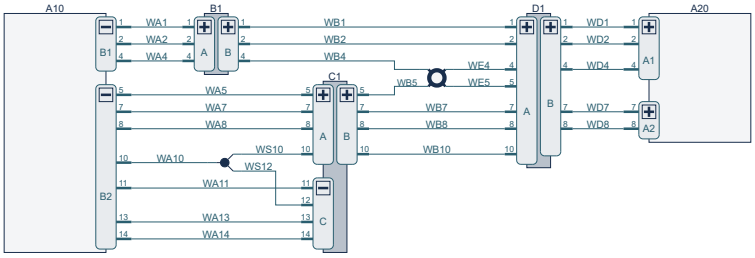


Figure 2: *Schematic generated by the example program using the C-API*

With a modified EDI configuration, that additionally (a) displays the Wire color (from Wire's “ color” attribute) and (b) displays labels from Wire's “Diameter” attributes and (c) displays the Component's background color (from Component's “ color” attribute) you should get this **picture** (or similar).

The Tcl Language API

Building the Containment Tree Structure

```
set edb [edb new]
$edb delete
```

The “`edb new`” command creates an empty database and returns a pointer to it. Technically, this pointer is a new Tcl command (e.g., *edb01*), which is used for further database related commands (in the following, we use $edb to refer to the current database).

The “`delete`” command deletes the database and its contents. After that, the $edb pointer to the database is invalid.

```
$edb new component       ?-name name? ?-ecu|-inliner|-splice|-eyelet|-svg?
$edb new connector $comp ?-name name? ?-anti? ?-male|-female|-half|-invisible?
$edb new cavity    $conn ?-name name? ?-halfdot|-spliced? ?-in? ?-out?
```

These commands create a new Component, Connector or Cavity object by calling NewComponent(), NewConnector(), or NewCavity() respectively, and return a pointer to the new object in form of an OID. All commands take an optional "`-name` *name*" argument which is directly passed to the respective C function call (`NULL` if omitted).

For the "`$edb new component`" command, the optional arguments (like "`-ecu`", "`-inliner`", etc) define the EdbComponentType, which is passed as third argument to the NewComponent() function (UNDEF if omitted).

The "`$edb new connector`" command takes a previously created Component *$comp* as second argument for the NewConnector() function call. The options are passed as EdbConnectorType (UNDEF if omitted).

The "`$edb new cavity`" command takes a previously created Connector *$conn* as second argument for the NewCavity() function call. The options are passed as EdbCavityType (UNDEF if omitted).

```
$edb new wire          ?-name name? ?-ground|-power|-logical|-bus|-hv|-arc?
$edb new multicore $mc ?-name name? ?-twisted|-shielded|-twshielded?
$edb new multicore 0   ?-name name? ?-twisted|-shielded|-twshielded?
```

These commands create a new Wire or Multicore object and return its OID. As before, they take an optional "`-name` *name*" argument which is directly passed to the respective C function call (`NULL` if omitted).

The given *$mc* must be either the OID of a previously created Multicore resulting in a node of the Multicore tree or "0" resulting in a top-level Multicore. The options are passed as EdbMulticoreType (UNDEF if omitted).

```
$edb new attr 0        name value
$edb new attr $comp    name value
$edb new attr $conn    name value
$edb new attr $cavity  name value
$edb new attr $wire    name value
$edb new attr $mc      name value
```

These commands add a new attribute (name-value pair) to the given object (or to the database root in case of "**0**") by calling the respective NewAttr4... function. Here, the commands pass the given OID as second, and *name* and *value* as third and fourth argument.

```
$edb validate attr name value
```

This command validates the given attribute (name-value pair) by calling the ValidateAttr() function. As a special case, calling `$edb validate attr " expr"` (without value) will validate all -config module's Boolean expressions.

Adding Connectivity

```
$edb join $cavity $wire
```

This command connects the given Cavity and Wire by calling the EdbJoin function.

```
$edb partner $conn    $conn
$edb partner $cavity  $cavity
```

Likewise, the "`$edb partner`" commands correlate two Inliner Connectors and Cavities, respectively, by calling the corresponding EdbPartner function.

Grouping Wires to a Multicore and connecting the Multicore shield

```
$edb group  $wire $multicore
$edb shield $wire $multicore
```

Both commands add a Wire to a Multicore forming a group of Wires by calling the EdbGroupWire function or the EdbShieldWire function respectively.

Defining Hierarchy

```
$edb hierarchy $comp
$edb hierarchy 0
```

This command changes the hierarchy where the following objects will belong to. It simply calls **`SetHierarchy()`**.

Defining Modules

```
$edb new module ?-name name? ?-function|-signal|-harness|-config|-always|-dbus?
$edb module add  $module $obj
$edb module addP $module $obj $obj2
$edb module addA $module $obj name value
$edb module addOption $module autocomplete
```

The "`new`" command creates a new Module by calling NewModule() and returns a pointer to the new module in form of an OID.

The family of "`module add`" commands add a reference to the given database object by calling the Add**Object**2Module() function. The OID returned from the "`new`" command is expected to be given as argument to the "`module add`" commands. The "`module addP`" command adds a reference to a pair of objects by calling the Add**ObjectPair**2Module() function. The "`module addA`" command adds a reference to an object and a name-value pair by calling the Add**ObjectAttr**2Module() function. The "`module addOption`" adds an option flag to the module; currently the only supported option is `autocomplete`. If this option is set, further objects are automatically added to the module after the 100% filtering has happened. See the Add**Option**2Module() function for more details.

The OID – Object ID

The OIDs are opaque to the API, however, they are necessary in order to connect database objects by passing them as arguments to other commands (similar to pointers). The lifetime of the OIDs coincides with the lifetime of the database. Like pointers, the OIDs are unique and cannot be exchanged with other databases.

To optionally define unique Object ID Numbers, the create Edb command edb new supports the option "`-maxid max`" and each of the $edb new component, new connector, new cavity, new wire, new multicore and new module support the option "`-id n`".

Saving/Restoring the EDB to/from a file

```
$edb save    ?-crypted $passwd $cipher? $filename
$edb restore ?-crypted $passwd?         $filename
```

The "`edb save`" command saves an already existing database to the given file name, "`edb restore`" restores a saved database from the given file into an existing edb. The edb is expected to be empty. Otherwise the restored data will be added to the edb.

Example Programs

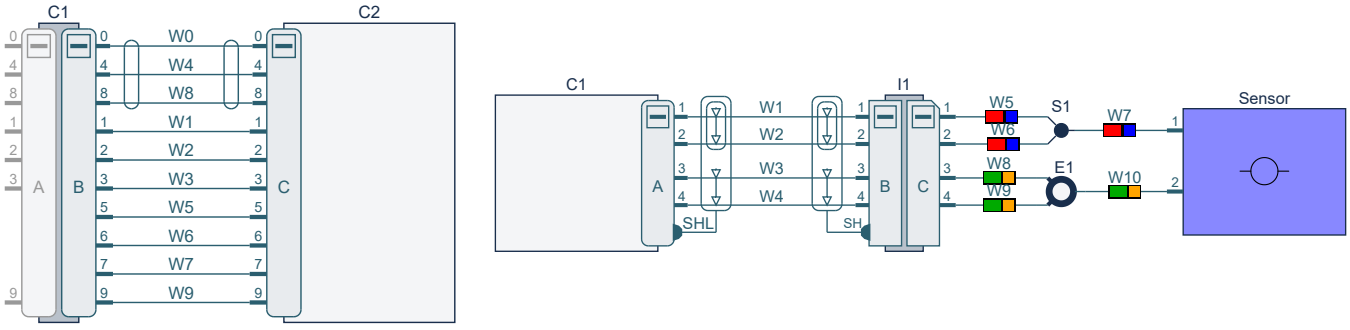


Figure 3: *The schematic generated by tapp1.tcl (left) and by tapp3.tcl (right), respectively.*

We offer two example programs `tapp1.tcl` and `tapp3.tcl` that create small Edb files using the Tcl EDB Creator API. The source code can be accessed by clicking on the links. The content of the resulting Edb files is displayed in the images in Figure 3.

Java Language API

The EDB Creator API does not come with native Java support, but the Java Native Access package can be used to access the C API. The API function prototypes needed for the Java Native Access package are available in JedbCreator.java. Please find some compile and usage hints in the README_JNA file.

Example Programs: There are Java example programs in Tapp1.java and Tapp3.java, that create the same small Edb files as the Tcl examples tapp1.tcl and tapp3.tcl above do.

The Python Language API

The EDB Creator API does not come with native Python support, but with a Python wrapper for the C API. The Wrapper code is available in PedbCreator.py.

Example Programs: There are Python example programs in Tapp1.py and Tapp3.py, that create the same small Edb files as the Tcl examples tapp1.tcl and tapp3.tcl above do.

The .NET C# Language API

The EDB Creator API does not come with native .NET support, but with a C# wrapper for the C API. The Wrapper code is available in NedbCreator.cs.

Example Programs: There are two .NET C# example programs Tapp1.cs and Tapp3.cs, that create the same small Edb files as the Tcl examples tapp1.tcl and tapp3.tcl above do.
